JavaScript: funções

declaração • parâmetros • retorno • escopo • organização do código

Prof. Dionizio Porfírio
Unichristus • 2026.2

Objetivos de aprendizagem

Ao final da aula, o estudante deverá ser capaz de:

- Declarar e chamar funções em JavaScript utilizando a sintaxe tradicional.
- Diferenciar parâmetros de argumentos e utilizar múltiplos parâmetros.
- Compreender quando uma função apenas executa uma ação e quando deve retornar um valor.
- Utilizar return para devolver resultados e encerrar a execução de uma função.
- Reconhecer o escopo básico de variáveis locais e externas.
- Aplicar funções a problemas com números, strings e arrays.
- Organizar o fluxo principal e as funções em arquivos JavaScript separados.
- Reconhecer a sintaxe básica de arrow functions.

Por que criar funções?

Funções permitem dar um nome a uma tarefa e reutilizar essa tarefa em diferentes pontos do programa.

Evitar repetição

Uma lógica escrita uma vez pode ser chamada quantas vezes forem necessárias.

Separar responsabilidades

O programa deixa de ser um bloco único e passa a ser dividido em pequenas tarefas.

Facilitar manutenção

Quando uma regra muda, a alteração fica concentrada na função responsável por ela.

A ideia é a mesma dos métodos que a turma já utilizou em Java: receber dados, processar e, quando necessário, devolver um resultado.

Anatomia de uma função

A declaração define a função. O código dentro das chaves só é executado quando a função é chamada.

```
function calcularDobro(numero) {
    let resultado = numero * 2;
    return resultado;
}
```

Nome da função

calcularDobro identifica a tarefa que poderá ser chamada depois.

Parâmetro

numero representa o dado que a função receberá durante a chamada.

return

Devolve o resultado para o ponto do programa que chamou a função.

Declarar não é o mesmo que chamar

A declaração ensina ao programa o que a função faz. A chamada solicita que essa tarefa seja executada.

1. Declaração

A função fica disponível para uso. Nenhuma mensagem será exibida apenas por declarar a função.

```
function saudar() {
    console.log("Olá!");
}
```

2. Chamada

Os parênteses após o nome indicam que a função deverá ser executada naquele momento.

```
saudar();
saudar();
```

Resultado: a mensagem será exibida duas vezes.

Parâmetros e argumentos

Parâmetros são nomes usados na definição da função. Argumentos são os valores enviados em cada chamada.

```
function saudar(nome) {
    console.log(`Olá, ${nome}!`);
}

saudar("Ana");
saudar("Carlos");
```

Parâmetro

nome é uma variável local criada para representar o valor recebido.

Argumentos

"Ana" e "Carlos" são valores diferentes enviados para a mesma função.

Uma função pode ter nenhum, um ou vários parâmetros.

Quatro combinações comuns

Parâmetros e retorno são decisões independentes. Nem toda função precisa receber dados ou devolver um valor.

Sem parâmetro • sem retorno

Executa uma ação fixa.
Ex.: mostrarMenu()

Com parâmetro • sem retorno

Executa uma ação usando um dado recebido.
Ex.: exibirMensagem(texto)

Sem parâmetro • com retorno

Produz um valor sem depender de argumento.
Ex.: gerarNumero()

Com parâmetro • com retorno

Recebe dados, processa e devolve resultado.
Ex.: calcularMedia(n1, n2)

Exemplo — função que executa uma ação

Crie uma função que receba o nome de um aluno e exiba uma mensagem de boas-vindas.

- A função deverá possuir um parâmetro chamado nome.
- A mensagem deverá ser construída utilizando template literal.
- A função não precisa retornar nenhum valor.
- Faça duas chamadas utilizando nomes diferentes.

Neste caso, o objetivo da função é produzir uma saída; por isso, console.log() pode ficar dentro dela.

Quando usar return

return envia um valor de volta para quem chamou a função. Esse valor pode ser armazenado, comparado ou utilizado em outra expressão.

```
function quadrado(numero) {
    return numero * numero;
}

let resultado = quadrado(5);
console.log(resultado);
```

A função calcula

quadrado(5) produz o valor 25.

Quem chamou decide

O resultado pode ser exibido, guardado ou reutilizado em outro cálculo.

Retornar um valor é diferente de apenas mostrá-lo no console.

Exemplo — calculando uma média

A função recebe duas notas e devolve a média. O fluxo principal fica responsável por ler os dados e exibir o resultado.

```
function calcularMedia(n1, n2) {
    return (n1 + n2) / 2;
}

let nota1 = Number(prompt("Nota 1:"));
let nota2 = Number(prompt("Nota 2:"));

let media = calcularMedia(nota1, nota2);
console.log(`Média: ${media}`);
```

Entrada

prompt() permanece fora da função.

Processamento

A função recebe apenas os valores necessários e retorna o cálculo.

return também encerra a função

Quando um return é executado, as instruções seguintes daquela função não são executadas. Isso pode simplificar validações.

```
function calcularDesconto(valor, percentual) {
    if (valor < 0 || percentual < 0) {
        return -1;
    }

    return valor * percentual / 100;
}
```

Entrada inválida

A função retorna -1 imediatamente e termina sua execução.

Entrada válida

A execução continua até o segundo return e devolve o desconto.

Múltiplos parâmetros: a ordem importa

Os argumentos são associados aos parâmetros pela posição em que aparecem na chamada.

```
function calcularTotal(quantidade, preco) {
    return quantidade * preco;
}

console.log(calcularTotal(3, 20)); // 60
console.log(calcularTotal(20, 3)); // 60
```

Atenção

Mesmo quando o resultado ocasionalmente é igual, trocar argumentos pode alterar completamente o significado da chamada.

Boa prática

Use nomes de parâmetros que deixem explícito o papel de cada valor recebido.

Escopo: onde a variável existe?

Uma variável declarada dentro de uma função pertence àquele contexto. Ela não fica automaticamente disponível no restante do programa.

```
function calcularDobro(numero) {
    let resultado = numero * 2;
    return resultado;
}

console.log(calcularDobro(10));
console.log(resultado); // erro
```

Variável local

resultado só existe dentro de calcularDobro().

Parâmetro também é local

numero também só existe durante aquela chamada da função.

Variáveis externas e funções

Uma função pode acessar uma variável definida fora dela, mas depender demais desse estado externo dificulta entender e testar o código.

```
let taxa = 0.10;

function aplicarTaxa(valor) {
    return valor + valor * taxa;
}

console.log(aplicarTaxa(100));
```

Funciona

taxa está disponível para a função porque foi declarada em um escopo externo.

Mas prefira dependências explícitas

Quando fizer sentido, receber a taxa como parâmetro torna a função mais independente.

Funções também podem processar strings

Uma função pode concentrar uma sequência de operações textuais e devolver o texto já normalizado.

```
function normalizarUsuario(texto) {
    texto = texto.trim().toLowerCase();
    texto = texto.replaceAll(" ", ".");
    return texto;
}

console.log(normalizarUsuario("  Ana Lima  "));
// ana.lima
```

Entrada

A função recebe uma string qualquer.

Saída

O return devolve uma nova string já transformada.

Funções também podem receber arrays

Arrays podem ser enviados como argumentos. A função pode percorrê-los e devolver um resultado calculado a partir dos elementos.

```
function contarAcima(valores, limite) {
    let quantidade = 0;

    for (let valor of valores) {
        if (valor > limite) {
            quantidade++;
        }
    }

    return quantidade;
}
```

Exemplo de chamada

contarAcima([5, 12, 8, 20], 10) retorna 2.

Reutilização

A mesma função funciona com arrays e limites diferentes sem alterar sua implementação.

Exemplo — maior valor de um array

Implemente uma função que receba um array numérico e devolva o maior valor encontrado.

- Considere que o array possui pelo menos um elemento.
- Utilize o primeiro elemento como valor inicial para a comparação.
- Percorra o array comparando cada elemento com o maior valor atual.
- A função deverá retornar somente o maior valor encontrado.
- Teste a função com pelo menos dois arrays diferentes.

Faça primeiro com for ou for...of. Não é necessário utilizar métodos de ordenação.

Uma função pode chamar outra função

Dividir uma regra em funções menores permite reaproveitar cálculos sem duplicar código.

```
function calcularMedia(n1, n2) {
    return (n1 + n2) / 2;
}

function verificarSituacao(n1, n2) {
    let media = calcularMedia(n1, n2);

    if (media >= 7) {
        return "Aprovado";
    }

    return "Reprovado";
}
```

calcularMedia()

Cuida apenas do cálculo.

verificarSituacao()

Reutiliza o cálculo e cuida da decisão.

Organizando funções em arquivos diferentes

No navegador, é possível carregar um arquivo com funções antes do arquivo que contém o fluxo principal.

funcoes.js

Contém as funções reutilizáveis.

```
function somar(a, b) {
    return a + b;
}
```

main.js

Utiliza as funções carregadas anteriormente.

```
let total = somar(10, 20);
console.log(total);
```

index.html

A ordem dos scripts importa.

```
<script src="funcoes.js"></script>
<script src="main.js"></script>
```

Para esta etapa da disciplina, não é necessário utilizar módulos, import ou export.

Retornar ou imprimir?

Uma função que retorna um valor costuma ser mais reutilizável, porque quem chama decide o que fazer com o resultado.

Apenas imprime

```
function somar(a, b) {
    console.log(a + b);
}

let x = somar(2, 3);
// x recebe undefined
```

Retorna o valor

```
function somar(a, b) {
    return a + b;
}

let x = somar(2, 3);
// x recebe 5
```

Regra prática: se o resultado será reutilizado pelo programa, prefira return.

Arrow functions — primeira leitura

JavaScript também possui uma sintaxe mais curta para escrever funções.

```
function somar(a, b) {
    return a + b;
}
```

```
const somar = (a, b) => {
    return a + b;
};
```

Função tradicional

É a forma principal desta aula.

Arrow function

Será especialmente importante na próxima aula, ao trabalhar com callbacks.

Erros comuns com funções

Esquecer os parênteses

somar referencia a função; somar() executa a função.

Confundir print com return

console.log() exibe. return devolve um valor ao chamador.

Trocar argumentos

A posição dos argumentos deve corresponder aos parâmetros.

Usar variável local fora

Variáveis declaradas dentro da função não existem automaticamente fora dela.

Esquecer return

Sem return explícito, uma função chamada como valor devolve undefined.

Fazer tudo em uma função

Funções menores e com uma responsabilidade são mais fáceis de entender.

Exercício 1 — valores acima da média

Desenvolva a função contarAcimaDaMedia(valores).

- Receba um array contendo valores numéricos e considere que ele possui pelo menos um elemento.
- Calcule a média de todos os valores utilizando uma estrutura de repetição.
- Percorra novamente o array e conte quantos elementos são estritamente maiores que a média calculada.
- Não altere o array recebido como parâmetro.
- Retorne a quantidade encontrada.

Exemplo: [4, 6, 8, 10] possui média 7 e dois valores acima da média.

Exercício 2 — geração de referência

Desenvolva a função gerarReferencia(nomeProduto, categoria, ano).

- Remova espaços externos de nomeProduto e categoria.
- Utilize somente os três primeiros caracteres da categoria, convertidos para letras maiúsculas.
- Separe o nome do produto em palavras e forme uma sigla com a primeira letra de cada palavra.
- Converta a sigla para letras maiúsculas.
- Retorne o resultado no formato CATEGORIA-ANO-SIGLA.

Exemplo: "Caderno Universitário", "papelaria", 2026 → PAP-2026-CU

Resumo da aula

Funções organizam o programa em tarefas nomeadas, reutilizáveis e mais fáceis de testar.

Declaração e chamada

function nome(...) { ... }
nome(...);

Parâmetros

Representam os dados que a função receberá em cada chamada.

return

Devolve um resultado e encerra a execução da função.

Escopo

Variáveis locais pertencem ao contexto em que foram declaradas.

Reutilização

Funções podem receber números, strings, arrays e resultados de outras funções.

Próxima aula

Funções como valores e callbacks; depois, forEach(), map(), filter() e outros métodos.